

Centro Paula Souza

Faculdade de Tecnologia de Ourinhos

Tecnologia em Análise e Desenvolvimento de Sistemas



Trabalho Prático Final

Analizador de Sequências de DNA

Linguagem de Programação — Linguagem C

Valor: 10,0 pontos • **Entrega:** 09/06/2026 • **Grupos:** 4 a 5 integrantes

Roteiro da Apresentação

1. Visão geral do trabalho
2. O que é DNA e por que é interessante
3. Divisão do trabalho (Básico / Intermediário / Avançado)
4. Requisitos de cada parte
5. Esqueleto fornecido
6. Entregáveis e avaliação
7. Cronograma e próximos passos

O que vamos construir?

Um **analisador de sequências de DNA** em Linguagem C.

O sistema vai permitir:

- Cadastrar sequências de DNA
- Contar bases (A, T, C, G)
- Calcular conteúdo GC
- Buscar padrões (motifs)
- Ordenar sequências
- Salvar e carregar de arquivo
- Comparar sequências

E vai exercitar:

- Variáveis e operadores
- if, switch, laços
- Funções e modularização
- Vetores e strings
- structs
- Ordenação e ponteiros
- Manipulação de arquivos

Por que o tema é DNA?

Bioinformática = programação aplicada à biologia

Empresas como **Illumina**, **Roche**, **Genentech** e laboratórios de pesquisa em todo o mundo precisam de programadores que saibam manipular sequências genéticas. É uma das áreas que mais cresce no mundo.

- **DNA é literalmente uma string** — só com 4 caracteres possíveis (A, T, C, G).
- **Os problemas são interessantes:** buscar padrões, comparar mutações, identificar genes. . .
- **É uma área concreta e demonstrável:** dá pra mostrar para qualquer pessoa o que o sistema faz.
- **Coloca no portfólio:** “desenvolvi um analisador de sequências genéticas em C”.

DNA: a molécula da vida

O DNA armazena a informação genética de todos os seres vivos.

É formado por uma sequência das **quatro bases**:

- **A** — Adenina
- **T** — Timina
- **C** — Citosina
- **G** — Guanina



```
char dna[] = "ATGCCGTAA";  
  
// E so isso!  
// Uma string com so 4  
// caracteres possiveis.
```

Em C, tratamos como uma string:

Conceitos biológicos que vamos usar

Conteúdo GC

Percentual de bases G + C em uma sequência. Diferentes organismos têm GC característico: humanos $\approx 41\%$, *E. coli* $\approx 51\%$.

Motif (padrão)

Um **padrão curto** de bases com significado biológico.

Exemplos: TATA (início de genes), GAATTC (enzima EcoRI), ATG (códon de início).

Distância de Hamming

Quantas posições diferem entre duas sequências de mesmo tamanho. Usada para detectar mutações.

Três níveis de dificuldade

Parte	Conteúdo	Peso
I	Básico — menu, cadastro, contagem, GC	6,0 pts
II	Intermediário — struct, ordenação, motif	2,0 pts
III	Avançado — arquivos, ponteiros, Hamming	2,0 pts
TOTAL		10,0 pts

60% fácil + 20% médio + 20% desafiador

Parte I — Básico (6,0 pts)

Objetivo: construir os fundamentos do sistema

O grupo vai implementar:

- **Menu principal** com do-while + switch
- **Cadastro** de até 10 sequências
- **Validação** (só A, T, C, G)
- **Listagem** formatada
- **Contagem** de cada base
- **Cálculo do GC**
- **Modularização** em funções

Conteúdos aplicados:

int, float, char[], operadores, if/switch, for/while/do-while, funções, vetores, strings.

Parte I — Exemplo de função esperada

Função para calcular o conteúdo GC:

```
float calcularGC(char *dna, int tamanho) {  
    int gc = 0;  
    for (int i = 0; i < tamanho; i++) {  
        if (dna[i] == 'G' || dna[i] == 'C') {  
            gc++;  
        }  
    }  
    return (float)gc / tamanho * 100.0;  
}
```

Saída esperada:

```
Sequencia Seq_A: GC = 52.63%
```

Parte II — Intermediário (2,0 pts)

Objetivo: organizar e ordenar

O grupo vai implementar:

- **Refatorar** usando `struct Sequencia`
- **Ordenar** sequências por tamanho (escolher 1 algoritmo: Bolha, Seleção ou Inserção)
- **Buscar motif** em uma sequência

Conteúdos novos aplicados:

`struct`, `typedef`, algoritmos de ordenação, busca de substring.

Parte II — A nova struct

Antes (vetores paralelos – ruim!):

```
char nomes[10][30];           // nome de cada sequencia
char dnas[10][200];           // DNA de cada sequencia
int tamanhos[10];             // tamanho de cada sequencia
// Dados relacionados ficam ESPALHADOS pelo codigo!
```

Depois (com struct – bom!):

```
typedef struct {
    char nome[31];
    char dna[201];
    int tamanho;
    float gc;
} Sequencia;

Sequencia banco[10];          // tudo junto e organizado!
```

Parte III — Avançado (2,0 pts)

Objetivo: persistência e comparação

O grupo vai implementar:

- **Salvar** sequências em arquivo (`sequencias.txt`)
- **Carregar** sequências do arquivo
- **Comparar** duas sequências usando **distância de Hamming**
- **Usar ponteiros** (obrigatório) nas funções de comparação

Conteúdos novos aplicados:

Ponteiros (`*`, `&`, `->`), manipulação de arquivos (`fopen`, `fprintf`, `fgets`, `fclose`).

Parte III — Ponteiros em ação

A função recebe *ponteiros* para as sequências:

```
int distanciaHamming(Sequencia *s1, Sequencia *s2) {
    if (s1->tamanho != s2->tamanho) {
        return -1;        // tamanhos diferentes
    }
    int distancia = 0;
    for (int i = 0; i < s1->tamanho; i++) {
        if (s1->dna[i] != s2->dna[i]) {
            distancia++;
        }
    }
    return distancia;
}
```

Atenção: acessar campos via ponteiro usa -> em vez de . .

Parte III — Arquivos

Formato simples do arquivo `sequencias.txt`:

```
Seq_A
ATGCCGTAACGGATTAGCC
Seq_B
GCGCGCGCATATATCGCGCG
Seq_C
AAAATTTTAAAATTTTAAAT
```

Sempre seguir os 4 passos:

1. `fopen` no modo correto ("`r`", "`w`" ou "`a`")
2. **Verificar se retornou** `NULL` (não pular!)
3. Ler/escrever com `fprintf`, `fgets`, `fscanf`, `fputs`
4. `fclose` ao final

O que vocês recebem prontos?

Vocês não começam do zero!

Disponibilizo um **esqueleto mínimo** com:

- **Estrutura de arquivos modular** (.c + .h separados)
- **Menu principal funcionando** (já roda, navega)
- **Assinaturas das funções** já definidas
- **Makefile** para compilação fácil (make, make run, make clean)
- **README** com instruções
- **Arquivo de exemplo** com sequências para testes

As funções principais estão como TODO — é o que vocês implementam.

Arquitetura do projeto

Arquivo	Responsabilidade
<code>main.c</code>	Menu principal (já vem pronto)
<code>sequencia.h</code>	Definição da struct Sequencia e constantes
<code>cadastro.c/.h</code>	Cadastro, listagem, validação (Parte I)
<code>analise.c/.h</code>	Contagem de bases, GC, motif (Partes I e II)
<code>ordenacao.c/.h</code>	Algoritmo de ordenação (Parte II)
<code>arquivo.c/.h</code>	Salvar / carregar arquivo (Parte III)
<code>comparacao.c/.h</code>	Distância de Hamming (Parte III)
Makefile	Automação de compilação

Modularização = código profissional.

Como compilar e rodar

Com Makefile (recomendado):

```
$ make          # compila o programa
$ make run      # compila e executa
$ make clean    # limpa arquivos gerados
```

Sem Makefile (compilação manual):

```
$ gcc -Wall -Wextra -std=c99 -o analisador_dna \
    main.c cadastro.c analise.c ordenacao.c \
    arquivo.c comparacao.c

$ ./analisador_dna
```

**Sempre compile com `-Wall -Wextra`
e elimine todos os warnings!**

O que vocês precisam entregar

1. Código-fonte completo

Arquivo TrabalhoDNA_GRUPO_X.zip com todos os .c e .h.

2. Relatório em PDF (4 a 6 páginas)

Capa, descrição, trechos de código comentados, tabela de testes, capturas de tela, divisão de tarefas, considerações.

3. Vídeo de apresentação (5 a 8 minutos)

Todos os integrantes participam, demonstrando o sistema funcionando e explicando o código.

Critérios de avaliação

Critério	Detalhamento	Peso
Parte I (Básico)	Menu, cadastro, contagem, GC	6,0
Parte II (Intermediário)	struct, ordenação, motif	2,0
Parte III (Avançado)	Arquivos, ponteiros, Hamming	2,0
TOTAL		10,0

Penalidades:

- Código não compila: **nota zero** na parte
- Plágio: **nota zero no trabalho inteiro**
- Entrega atrasada: $-1,0$ por dia (limite 3 dias)
- Vídeo sem algum integrante: $-1,0$ para o ausente

Cronograma sugerido (2 semanas)

Dia	Etapa	Atividades
1–2	Planejamento	Reunião, divisão de tarefas, leitura do esqueleto
3–7	Parte I	Cadastro, listagem, contagem, GC
8–10	Parte II	struct, ordenação, motif
11–12	Parte III	Arquivos e Hamming
13	Documentação	Relatório, capturas, vídeo
14	Entrega	Testes finais e upload

Não deixem tudo para a última semana!

A Parte III demanda tempo. Comecem cedo.

Divisão de tarefas sugerida

Integrante	Responsabilidade principal
1	Menu principal + integração entre módulos
2	<code>cadastro.c</code> + <code>analise.c</code> (contagem, GC)
3	Algoritmo de ordenação + busca de motif
4	Manipulação de arquivos (salvar / carregar)
5	Distância de Hamming + relatório / testes

Importante: todos devem entender o código todo!
A divisão é só para quem escreve *primeiro* cada parte.

5 dicas para ter sucesso

1. **Compilem com frequência**

Não escrevam 500 linhas e depois tentem compilar. A cada função, `make`.

2. **Testem cada função isoladamente**

Implementou `contarBases`? Cadastra uma sequência e testa antes de seguir.

3. **Usem o Git**

GitHub, GitLab, ou qualquer um. Evita perda de código e facilita o trabalho em grupo.

4. **Validem entradas do usuário**

Nunca confiem que o usuário vai digitar algo certo. Sempre validem.

5. **Comentem o código**

O “eu de daqui a 3 dias” não vai lembrar o que “eu de hoje” escreveu.

Erros comuns para evitar

- **Esquecer o** `break` no `switch` (cai no próximo case!)
- **Acessar índice fora do vetor** — causa crash ou bug bizarro
- **Esquecer o** `&` no `scanf` para variáveis simples
`scanf("%d", &n)` — **não** `scanf("%d", n)`
- **Esquecer de fechar arquivo** com `fclose` — perde dados
- **Não verificar** `NULL` retornado por `fopen` — crash
- **Confundir** `*` e `&` em ponteiros — estudem com calma
- **Esquecer o** `'\0'` ao montar strings manualmente

Sequências de teste recomendadas

Seq_A:	ATGCCGTAACGGATTAGCC	(19 bases - mista)
Seq_B:	GCGCGCGCATATATCGCGCG	(20 bases - rica em GC)
Seq_C:	AAAATTTTAAAATTTTAAAAT	(21 bases - rica em AT)
Seq_D:	ATGCCGTAACGGATTAGCC	(19 bases - idêntica a A)
Seq_E:	ATGCCGTAACGGATTAGCT	(19 bases - 1 dif. de A)

Resultados esperados:

- **Seq_A:** A=5, T=4, C=5, G=5, GC=52.63%
- **Seq_B:** A=2, T=2, C=8, G=8, GC=80.00%
- **Seq_C:** A=12, T=8, C=0, G=0, GC=0.00%
- **Hamming(A, D)** = 0 (idênticas)
- **Hamming(A, E)** = 1 (uma diferença)

Resumindo

Em 2 semanas, vocês vão entregar:

Tecnicamente:

- Um analisador funcional
- Que cobre todo o conteúdo de C
- Modularizado e bem comentado
- Com plano de testes preenchido

Como aprendizado:

- Domínio de struct e ponteiros
- Experiência com modularização
- Trabalho em equipe com Git
- Algo concreto pro portfólio

Lembrem:

A nota não é o objetivo — é a **consequência** de aprender de verdade. Façam pensando em entender, não em “passar”.

Dúvidas?

Procurem o auxiliar docente André Mendes.

Bom trabalho!

A T C G